GLYVEX

Touchpad Gesture Recognition & Command Execution

Draw a symbol. Let the machine understand it. Turn it into an action.

01 PROJECT DOSSIER

Concept. A Java desktop system continuously monitors a laptop touchpad, recognizes a deliberate four-corner activation sequence, captures a handwritten symbol, classifies it with machine learning, and maps the result to a predefined computer action.

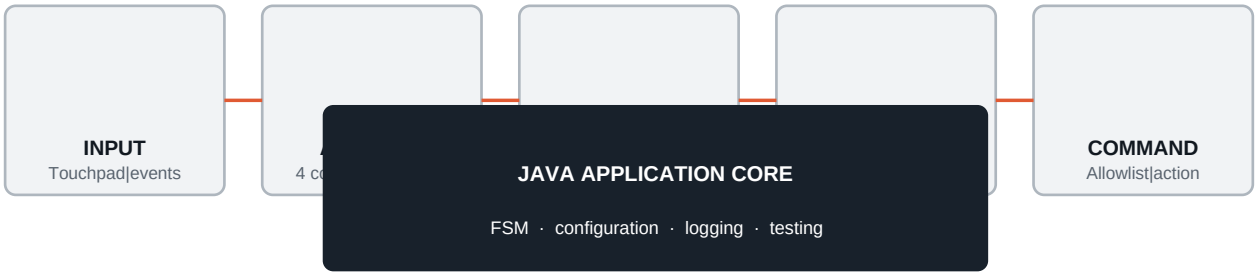
Engineering domains. Human–Computer Interaction · Touch Input · Gesture Processing · Machine Learning · Pattern Recognition · Java Desktop Development · Controlled Automation

Registration No.	Team member	Email
AP25110010712	Ashwath Raj Mulakalapalli	ashwath_m@srmap.edu.in
AP25110010120	BARATAM PRANEETH GUPTA	praneeth_baratam@srmap.edu.in
AP25110010905	Harsha Vardhan Kumar	harsha_jogi@srmap.edu.in
AP25110010841	Sai thanush reddy	thanush_bobbili@srmap.edu.in

Java · ML dataset · Physical touchpad · Local command execution

02 The Idea in One Page

GLYVEX is not simply a handwriting recognizer. Its central idea is a complete interaction loop: the user deliberately unlocks a hidden gesture surface, draws a symbol, and receives a desktop action.



The four-corner sequence is the activation gate. It separates ordinary touchpad activity from intentional command input. Once activated, the touchpad temporarily behaves like a drawing canvas.

The ML model has a deliberately narrow responsibility: convert a normalized trajectory into one of a controlled set of gesture classes. The command system then decides what that class is allowed to do.

> **Example:** four corners → draw **C** → classifier predicts **OPEN_CALCULATOR** → command registry launches Calculator.

The result is a clear demonstration story: **physical sensing** → **intelligent interpretation** → **software action**.

03 Problem Statement & Motivation

Traditional desktop commands are usually tied to keyboard combinations, visible controls or fixed gestures. A laptop touchpad is already a high-resolution input surface, but it is rarely treated as a customizable command canvas.

GLYVEX investigates whether a deliberate spatial sequence followed by a handwritten symbol can become a practical desktop interaction mechanism.

The engineering problem is broader than classification. The application must acquire touch events, distinguish activation from drawing, segment the trajectory, normalize the data, classify the gesture, reject uncertainty and safely execute the selected action.

Why four corners? A single drawing could be triggered accidentally. A deliberate sequence adds an activation boundary.

Why ML? Users do not draw identical shapes. A classifier can tolerate natural variation in size, position, speed and style.

Why Java? Java provides the main desktop application, state management, UI, testing, configuration and local inference environment required by the project.

04 Objectives, Scope & Success Criteria

Objective	Success condition
Touch monitoring	Supported touchpad events are acquired reliably.
Activation	Four-corner sequence changes the state consistently.
Capture	Complete drawings are stored as trajectories.
Preprocessing	Different instances become comparable representations.
Recognition	Supported classes are predicted with measurable performance.
Safety	Uncertain predictions do not trigger commands.
Execution	Only configured benign actions are launched.

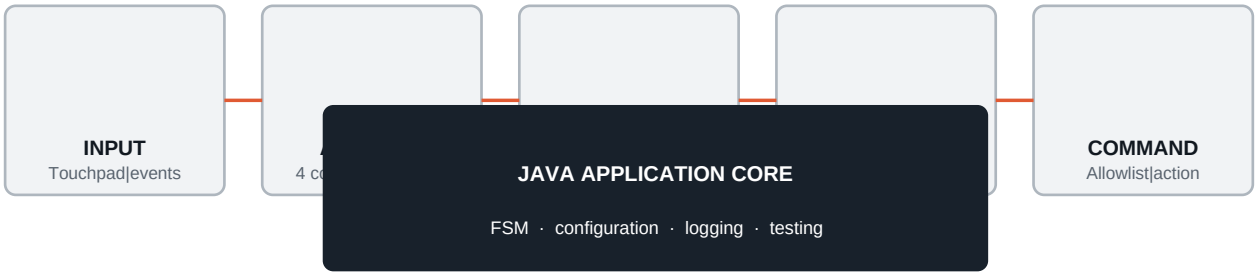
Scope boundary. The first version should support a small command vocabulary. Universal handwriting recognition, unrestricted shell commands, cloud inference and privileged automation are outside the core project.

A smaller system is easier to train, evaluate and defend in a viva because the team can explain exactly what each gesture means and why errors occur.

The project is complete when the full pipeline can be demonstrated on physical hardware and each major subsystem can also be tested independently.

05 System Architecture

The architecture deliberately separates hardware-specific code from the ML and command layers. This keeps the application maintainable and makes the most uncertain component—the touchpad interface—replaceable.



Input Adapter converts operating-system events into a Java-facing event model. **State Machine** determines whether an event belongs to activation, drawing or normal monitoring. **Trajectory Processor** standardizes the raw data. **ML Inference** predicts a class. **Command Router** validates and resolves that class before execution.

The same separation improves testing: recorded trajectories can test preprocessing and ML without hardware, while synthetic predictions can test the command layer without launching the complete interaction.

Data flow: Touch event → activation → capture → preprocessing → prediction → confidence check → command lookup → execution.

06 Interaction Flow

Stage	User action	System behavior
Monitoring	Normal laptop use	Events observed; no command active.
Activation	Tap four corners	Sequence progress is tracked.
Gesture Mode	Begin drawing	Trajectory recording begins.
Capture end	Release / timeout	Trajectory is finalized.
Recognition	No extra action	Features are generated and classified.
Decision	No extra action	Class and confidence are checked.
Execution	No extra action	Approved command is launched.
Recovery	Continue working	System returns to Monitoring.

The user never needs a permanent drawing window. The touchpad becomes the temporary interaction surface only after deliberate activation.

The system should provide concise feedback so the user knows whether the corner sequence was accepted, whether drawing is active and whether a command was rejected.

A failed gesture should be boring: cancel, return to Monitoring, and do nothing else. That is preferable to unpredictable behavior.

07 Finite-State Machine

A finite-state machine is preferable to scattered boolean flags. It defines exactly which transitions are valid and gives every failure a recovery path.

State	Purpose	Exit conditions
IDLE	Normal monitoring	Valid first corner
ARMING	Track corner sequence	Completion, timeout, wrong corner
GESTURE_MODE	Ready for drawing	Drawing begins / cancel
CAPTURING	Collect trajectory	Release / timeout
PROCESSING	Preprocess + classify	Accepted / rejected
EXECUTING	Run validated action	Success / failure
ERROR	Recover safely	Reset to IDLE

```
enum State {
IDLE, ARMING, GESTURE_MODE,
CAPTURING, PROCESSING, EXECUTING, ERROR
}
```

Timeouts are important in ARMING and CAPTURING. Without them, the application can remain waiting for an event that will never arrive.

08 Touchpad Input Engineering

This is the project's main feasibility risk. Standard JavaFX or Swing APIs do not universally expose low-level touchpad coordinates and contact events. GLYVEX should therefore define an input abstraction rather than coupling the application to one operating-system API.

```
interface TouchInputSource {
void start(InputListener listener);
void stop();
}

record TouchSample(double x, double y, long timeNanos) {}
```

Approach	Use when
Native OS bridge	The target OS exposes the required touch events.
Helper process + IPC	Native access is easier outside the JVM.
Java-only input	Only after real touchpad coordinates are verified.

Feasibility gate: prove raw coordinate acquisition before polishing the UI or finalizing the ML pipeline. A classifier cannot compensate for an input layer that cannot supply the required data.

The final report should name the target OS, event source, driver/interface and Java integration mechanism.

09 Four-Corner Activation

The touchpad is divided into four conceptual corner regions. Each region has a tolerance radius or percentage of the touchpad dimensions.

Parameter	Purpose
Corner tolerance	How close a tap must be to a corner.
Inter-tap timeout	Maximum delay between sequence taps.
Debounce interval	Prevents one tap from being counted twice.
Gesture timeout	Maximum drawing duration.
Sequence order	Initial prototype can use a fixed order.

A deterministic first version might use top-left → top-right → bottom-right → bottom-left. The order can later become configurable.

A partial sequence must expire. If the user taps two corners and then waits, the system should return to IDLE rather than remaining armed indefinitely.

Important: four-corner activation changes state; it does not itself execute a desktop command.

10 Gesture Capture & Data Representation

Once Gesture Mode is active, the touchpad becomes a temporary drawing surface. Every valid movement contributes to an ordered trajectory.

```
Trajectory = [(x■,y■,t■), (x■,y■,t■), …, (x■,y■,t■)]
```

Signal	Derived information
x, y	Position and geometric shape
Δx, Δy	Movement direction
Δt	Timing and speed
Bounding box	Scale and aspect ratio
Path length	Overall movement magnitude
Direction sequence	Local trajectory pattern

The timestamp is useful because two users can draw the same shape at very different speeds. Depending on the final model, timing can either be used as a feature or normalized away.

Raw data should be preserved long enough for debugging and replay. The ML layer should receive only the standardized representation it expects.

11 Preprocessing Pipeline

Raw touch trajectories are irregular. The same symbol may be drawn in different positions, at different sizes and with different numbers of samples.

1. Normalize. Map coordinates to a stable range.

2. Translate. Remove irrelevant absolute position.

3. Scale. Reduce variation in drawing size.

4. Resample. Convert variable-length trajectories into a fixed representation.

5. Smooth, if necessary. Reduce sensor jitter without destroying sharp turns.

6. Extract features. Build the representation required by the classifier.

```
raw touch samples
↓
normalize → center → resample → features
↓
model-ready vector
```

Training and inference must use exactly the same preprocessing. Otherwise the model is effectively trained and deployed on different input spaces.

12 Dataset & Data Collection

The dataset determines how credible the ML portion of the project will be. A large image-based handwriting dataset is not automatically equivalent to touchpad trajectories.

The most useful data contains actual trajectories from the target interaction. Multiple users should contribute examples so the model sees natural variation in drawing style.

Dimension	Recommended design
Classes	Begin with approximately 6–10 command symbols.
Users	Use multiple contributors.
Samples	Repeated examples for every class and user.
Variation	Different speed, size and starting position.
Test split	Keep unseen examples; ideally unseen users.
Metadata	Store class and capture/session information.

A user-independent test set is especially useful because it measures whether the model recognizes the gesture concept rather than one person's drawing habits.

The final report should document the dataset source, collection procedure, number of classes, sample counts, preprocessing and split methodology.

13 Machine-Learning Strategy

The ML task is multiclass classification. Each supported symbol represents one command class. Model complexity should be justified by the data rather than selected simply because a neural network sounds more advanced.

Candidate	Strength	Trade-off
k-NN	Excellent simple baseline	Inference depends on training-set size
Decision Tree	Interpretable	Can overfit
Random Forest	Strong classical baseline	Larger model
SVM	Good compact-space classifier	Needs tuning
Small MLP	Nonlinear representation	More training care
Sequence model	Uses temporal structure	Higher complexity

Recommended path: establish a baseline → inspect errors → improve representation/data → compare candidates → select the final model using validation results.

The report should explain why the chosen model fits the trajectory representation and command vocabulary.

14 Model Evaluation & Error Analysis

Accuracy alone is insufficient for a system that can launch actions. The project should measure recognition quality and uncertainty handling.

Metric	Question answered
Accuracy	How often is the prediction correct overall?
Precision	How reliable is each predicted class?
Recall	How often is each actual class detected?
F1-score	How well are precision and recall balanced?
Confusion matrix	Which symbols are confused?
Latency	How quickly does recognition complete?
Rejection rate	How often is uncertain input safely rejected?

Error analysis should investigate visually similar symbols and determine whether errors arise from poor normalization, insufficient data, ambiguous class definitions or model choice.

Example: if two classes have nearly identical shapes, collecting more unrelated samples may not help. The class design or discriminative features may need to change.

15

Java Technology Stack

Layer	Technology	Purpose
Language	Java 17+ / approved JDK	Core application
UI	JavaFX	Configuration and status
Build	Maven or Gradle	Dependencies/builds
Testing	JUnit 5	Automated testing
Logging	SLF4J + Logback	Diagnostics
Configuration	JSON / YAML / properties	Mappings/thresholds
Serialization	Jackson / Gson	Structured data
Inference	ONNX Runtime Java or equivalent	Local model execution
Packaging	jpackage	Desktop distribution
Version control	Git + GitHub	Collaboration

Why JavaFX? The UI requirement is compact and desktop-oriented. JavaFX keeps configuration and status inside the Java application.

Why local inference? It avoids network latency and keeps touch trajectories on the local machine.

Why an input adapter? Hardware access is the one area where platform-specific code may be unavoidable.

16 Software Structure

```
com.glyvex
■■■ app/ lifecycle
■■■ input/ touchpad adapters
■■■ gesture/ activation + FSM
■■■ processing/ normalization + features
■■■ ml/ model + inference
■■■ commands/ allowlist + execution
■■■ config/ settings
■■■ ui/ JavaFX interface
■■■ logging/ diagnostics
■■■ test/ unit + replay tests
```

app owns startup. **input** hides OS details. **gesture** owns interaction state. **processing** transforms samples. **ml** knows the model. **commands** is the execution boundary. **ui** exposes setup and feedback.

The separation also supports parallel teamwork. One member can improve preprocessing while another works on the input adapter without editing a single giant application class.

Dependency rule: the command executor should not know anything about raw touch events, and the touchpad adapter should not know which application a gesture launches.

17 Command Execution & Safety

The classifier should return a semantic command identifier rather than executable text.

```
Prediction: OPEN_BROWSER
↓
CommandRegistry.lookup("OPEN_BROWSER")
↓
Allowlist validation
↓
Configured Java launcher
↓
Browser opens
```

Control	Purpose
Allowlist	Defines which commands can exist.
Confidence threshold	Rejects uncertain predictions.
Configuration validation	Rejects malformed mappings.
Failure handling	Returns safely to IDLE.
Emergency disable	Stops gesture-triggered actions.
Logging	Records semantic events for debugging.

For the semester demonstration, benign commands such as opening a calculator, text editor, browser or predefined folder are enough to prove the concept.

18 User Interface & Experience

The UI should support the touch interaction rather than compete with it. The permanent interface is mainly for configuration, feedback and development.

Status: Monitoring, Armed, Gesture Mode, Recognizing, Executing or Cancelled.

Mapping: supported symbol → configured command.

Thresholds: corner tolerance, timeouts and confidence threshold.

Diagnostics: prediction/confidence and optional gesture preview during development.

Replay: feed stored trajectories into preprocessing and ML without physical input.

State	Feedback
Monitoring	Neutral monitoring indicator
Arming	Corner progress
Gesture Mode	Clear drawing prompt
Processing	Recognition indicator
Executing	Command/result
Cancelled	Short reason; return to monitoring

The final presentation should show the UI briefly and spend most of the demonstration time on the physical interaction and recognition pipeline.

19 Development Plan & Milestones

Development order matters. The team should de-risk hardware input before spending time polishing the interface.

Milestone	Focus	Output
M1	Input feasibility	Touch events captured
M2	Activation	Four-corner FSM
M3	Capture	Recorded trajectories
M4	Preprocessing	Stable representation
M5	Dataset	Labeled samples/splits
M6	Baseline ML	First classifier
M7	Model selection	Evaluation + confusion matrix
M8	Java inference	Local prediction
M9	Command layer	Allowlisted actions
M10	UI	Configuration/status
M11	Integration	End-to-end prototype
M12	Submission	Testing + report + demo

Critical dependency: M1 must succeed before the project assumes the physical touchpad can provide usable raw trajectories.

20 Testing Strategy

Testing should mirror the architecture. Each subsystem should be testable without requiring the entire application.

Input tests verify event conversion, timestamps and coordinates.

State tests verify valid/invalid transitions, timeouts and resets.

Processing tests verify deterministic feature generation.

ML tests verify model loading, class ordering and prediction behavior.

Command tests verify allowlisting without launching real applications.

Integration tests replay recorded trajectories through the complete pipeline.

User tests measure activation and gesture behavior across multiple users.

```
Recorded trajectory
↓
Preprocessor
↓
Feature vector
↓
Model
↓
Expected class
↓
Regression test
```

A trajectory replay harness is one of the highest-value development tools because it removes hardware variability from most ML and processing tests.

21 Resources & Development Environment

Resource	Role
Touchpad-equipped laptop	Physical input and final demonstration
JDK 17+	Java development/runtime
IntelliJ IDEA / VS Code / Eclipse	IDE
Maven / Gradle	Build management
JavaFX	Desktop interface
JUnit 5	Testing
Git + GitHub	Version control
Python + ML library, if permitted	Training/data experiments
ONNX Runtime Java or equivalent	Local inference
jpackage	Application packaging

Training and deployment environments should be documented separately. Python may be useful during model development if permitted by the course, while the final delivered application remains Java.

Repository discipline: source, configuration, documentation and small replay samples belong in version control. Large datasets and model artifacts should be handled deliberately.

Reproducibility: record major dependency versions, dataset versions, preprocessing settings and model metadata.

22 Risks, Limitations & Mitigation

Risk	Why it matters	Mitigation
No raw touch access	Core interaction fails	Prove input bridge first
False activation	Unexpected state changes	Four-corner gate + timeout
Missed activation	Poor UX	Tolerance + debounce
Class confusion	Wrong command risk	Confidence threshold + better data
Dataset leakage	Misleading evaluation	User-independent split
Slow inference	Interaction feels delayed	Compact model + local runtime
Command failure	Poor recovery	Validation + safe reset
State bug	Application can get stuck	Explicit FSM + timeouts

The two major technical uncertainties are hardware access and dataset quality. Both should be addressed early rather than hidden until final integration.

The final report should state limitations explicitly. A documented constraint is preferable to a claim that the prototype is universally reliable.

23 Team Structure & Responsibilities

Member	Primary area	Concrete responsibilities
Ashwath Raj Mulakalapalli	Architecture + Java integration	Project structure, FSM, Java interfaces, integration
BARATAM PRANEETH GUPTA	ML + data	Dataset, preprocessing, training, evaluation
Harsha Vardhan Kumar	Input subsystem	Touchpad bridge, corner detection, trajectory capture
Sai thanush reddy	UI + testing + documentation	JavaFX configuration, test harness, report/demo assets

This is a suggested work allocation. All members should understand the complete pipeline for the final viva.

Collaboration rule: agree on module interfaces early. The ML team should be able to use recorded trajectories while the hardware integration is still being refined.

Submission discipline: maintain meaningful commits, record experiments and document important design decisions.

24

Final Project Definition

GLYVEX is a Java-based touchpad interaction system that transforms deliberate physical gestures into desktop commands. The application begins in passive monitoring mode. The user activates the hidden command surface by tapping all four touchpad corners in the configured sequence.

After activation, the user draws a predefined symbol. The application records the trajectory, preprocesses it into a standardized representation, applies a machine-learning classifier and checks the result against a confidence threshold.

The recognized class is then resolved through an explicitly configured command registry. Only approved actions can be executed. After the command finishes—or is rejected—the application returns to monitoring.

> **GLYVEX in one line:** Four-corner activation → draw a glyph → recognize it with ML → execute a controlled command.

The project demonstrates a complete chain from **physical sensing** to **machine interpretation** to **software action**. Its strength should come from reliable integration, reproducible data processing and safe execution rather than unnecessary feature breadth.

Team: Ashwath Raj Mulakalapalli · BARATAM PRANEETH GUPTA · Harsha Vardhan Kumar · Sai thanush reddy

Core stack: Java · Touchpad Input Integration · Gesture FSM · ML Classification · Local Inference · JavaFX · JUnit · Git